GOSIM

FlexKV

面向大规模 LLM 推理的分布式 KVCache 管理框架

分级存储 · 池化共享 · 引擎中立



腾讯云 IaaS前沿 孙飞强 | GOSIM Shenzhen 2026

CONTENTS

GOSIM Shenzhen 2026

Part 01 背景：Agentic AI 时代，KVCache 为什么成了瓶颈

Part 02 FlexKV 的核心能力

Part 03 生态与集成

Part 04 总结：一套完整的 KVCache 管理方案



01

背景：Agentic AI 时代，KVCache 为什么成了瓶颈

以空间换时间 | Agent 的三个新特点 | HBM 装不下了



KVCache 基本原理：以空间换时间

Prefill 是大头

每个输入 token 都要计算 K/V，代价与上下文长度成正比

缓存复用

把已计算的 K/V 存下来，相同前缀直接读取，跳过重复计算

收益

多轮对话只需增量计算新 token，TTFT 显著下降、算力省下来

多轮对话请求的两种处理方式

无 KVCache



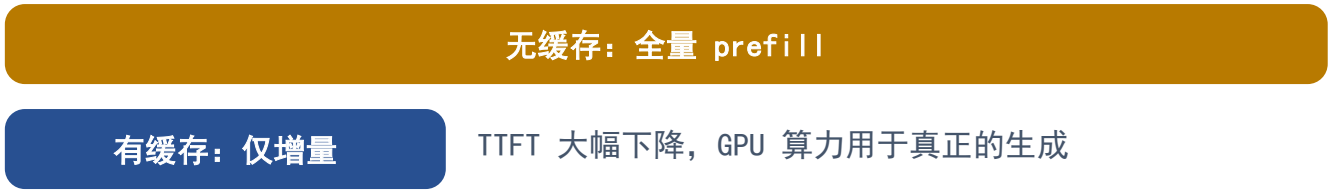
→ 每一轮都全量 prefill，前缀被反复重算

有 KVCache



→ 命中前缀直接复用，仅增量计算新 token

TTFT 对比（首 token 延迟）



◆ 前缀复用率越高，KVCache 收益越大 —— 而这正是 Agent 场景的常态

Agent 场景的三个新特点

01 上下文越来越长

- **长文档、代码仓库、工具结果**进入上下文
- 单请求可达**数十万 token**
- prefill 开销随长度**线性膨胀**

02 前缀高度复用

- **System Prompt + 工具定义 + 历史轮次**
- **多个子 Agent** 继承同一段父级前缀
- **同一模板**被海量会话与子任务共享
- 相同前缀反复出现，被**大量重复计算**

03 并发会话多

- **数百上千会话**并行，缓存争抢激烈
- 多会话**共享相同前缀**（同模板 / 同文档）
- **命中率**直接决定吞吐与成本

◆ **三个特点叠加：重复计算大规模存在，KVCache 复用的价值被急剧放大**

HBM 装不下爆发增长的 KVCache

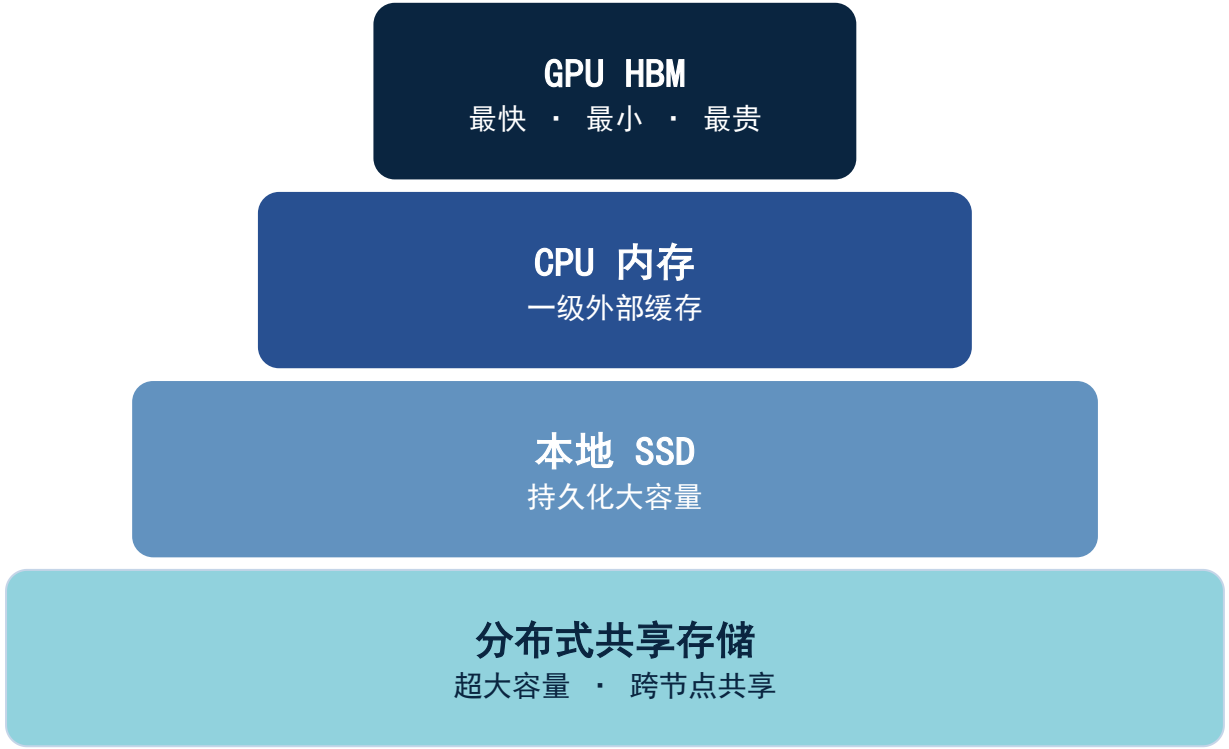
矛盾的根源

KVCache 随上下文长度与并发数急剧膨胀

HBM 容量有限，还要装模型权重与激活

装不下 → 淘汰 → 下次请求重新 prefill

后果：TTFT 恶化、GPU 算力被重复 prefill 浪费、单请求成本上升



▲ 自上而下：速度递减、容量递增、单位成本递减

◆ **“分级存储 + 池化共享”是必然方向：用低成本介质扩容量，用共享提命中率**

02

FlexKV 的核心能力

引擎中立的 KVCache 管理框架：统一管理 · 压缩 · 池化 · 高效传输 · 隔离



FlexKV 总体架构：控制面 + 存储面 + 数据面

推理引擎　vLLM / SGLang / TensorRT-LLM / Dynamo

put / get 异步 API

FlexKV（进程内库调用）

GlobalCacheEngine · 控制面	StorageEngine · 存储面	TransferEngine · 数据面
RadixTree 前缀匹配（match / insert） Mempool 空间追踪与淘汰触发 决策：从哪级取、取哪些 block	按 block 存储，KV shape 与 GPU 一致 CPU / SSD / 可扩展存储统一初始化 block-wise 模式合并更大 I/O 单位	多进程多线程并行传输 io_uring 加速 SSD I/O GDS 直传 / RDMA 跨节点

◆ 原生 Prometheus 监控 + 纯库函数形态（v1.0 API），无 client-server 通信开销

多级存储管理：一套索引调度 GPU 之下的所有介质

CPU 内存
一级外部缓存 · 微秒级

本地 SSD
持久化缓存 · 支持多盘并行

分布式扩展存储
集群级容量 · 跨节点共享（Mooncake Store）

Block 粒度存储

多 token 组成 block，KV shape 与 GPU 完全一致，地址即 block ID

统一前缀索引

每级介质一棵 RadixTree，跨级比较命中长度，取最优来源

灵活淘汰策略

LRU / LFU / SLRU / FIFO / MRU / FILO 六种 + 频率感知奖励；逻辑淘汰零数据搬

容量即配置

cpu_cache_gb / ssd_cache_gb 一行配置完成扩容，HugePage 降低 TLB 开销

KVCache 压缩：让同样的存储装下更多 KV



压缩在 D2H 路径完成：先压缩再落盘；解压在 H2D 路径完成，全程 GPU 端批量执行

nvCOMP ANS 无损压缩

基于 NVIDIA nvCOMP 的 ANS 熵编码，数据零失真

GPU 端批量压缩 / 解压

双缓冲流水线

压缩与传输双缓冲重叠执行，隐藏压缩开销

写 CPU/SSD 的数据量同步下降，带宽压力更小

低精度 KV 也无损

vLLM Blackwell NVFP4 KV Cache 字节级无损卸载与恢复

量化场景下缓存数据同样可靠

◆ 同样容量的 CPU / SSD 装下更多 KV，等效于容量和带宽双扩展

分布式 KVCache 池化：跨节点共享 + 单机多实例复用

单机单实例 → 单机多实例 → 跨节点集群池

KVCache 复用率高 → 池子越大，命中率越高

性能提升与成本优势同步放大 —— 池化是必然趋势

跨节点共享（集群级池化）

- **分布式 RadixTree 索引**
  每节点持有全局索引本地快照，查询无中心瓶颈、无网络往返
- **租约机制（Lease）**
  跨节点传输期间锁定数据，保证读到的一定有效
- **索引 Upload & Rebuild**
  本地索引定期上传 GMS（Redis），再拉取重建全局视图
- **Mooncake Transfer Engine**
  RDMA 零拷贝跨节点传输
- **Mooncake Store**
  基于 Key 的远端缓存层，实现集群级 KV 复用

单机多实例复用

- **共享同一套缓存**
  多个推理实例共用一份 CPU / SSD 缓存池
- **Server 自动创建**
  实例 0 创建 FlexKV Server，其余实例即插即用
- **跨实例前缀复用**
  相同前缀跨实例命中，整体命中率显著提升
- **消除缓存孤岛**
  单机多 GPU 部署多服务时，缓存不再各自为政

◆ 把分散在各节点、各实例的缓存变成一个可统一调度的池

高效传输：异步重叠 · 逐层流水 · 自适应选路

异步传输与计算重叠

- **get 预取**与当前计算重叠执行
- **put 下沉**与后续计算重叠
- 下层介质间传输，**主进程完全不感知**

逐层传输（Layerwise）

- KV **按层流水线** H2D 恢复
- **前几层就绪**即可开始计算
- 首 token 更早返回，**TTFT 进一步降低**
- 8×GPU 实测：批完成通知 **4.39 → 1.87 ms（2.35×）**

自适应路径选择

机内传输

- **Kernel / CE** 多模式自适应选择
- **PCIe 多路径**条带化并发
- **NUMA 亲和**的存储与传输

机间传输

- **RDMA 多路径**并发传输

硬件级加速

- **GDS**：SSD 直达 GPU，绕过 CPU
- **io_uring**：高并发 SSD I/O
- **RDMA**：跨节点零拷贝传输

Namespace 缓存隔离：多租户共池互不干扰

$$hash = H(\ namespace_id \ \|\ token_ids\ )$$

同一份 token，不同 namespace 哈希不同 —— 隔离发生在块哈希级别，零额外开销

多租户隔离

tenant / user 维度划分命名空间
不同租户缓存互不可见，安全与公平兼得

多模型 / 多 LoRA 共池

lora_name 或模型标识并入 namespace
多模型、多适配器共池，互不污染

会话级隔离

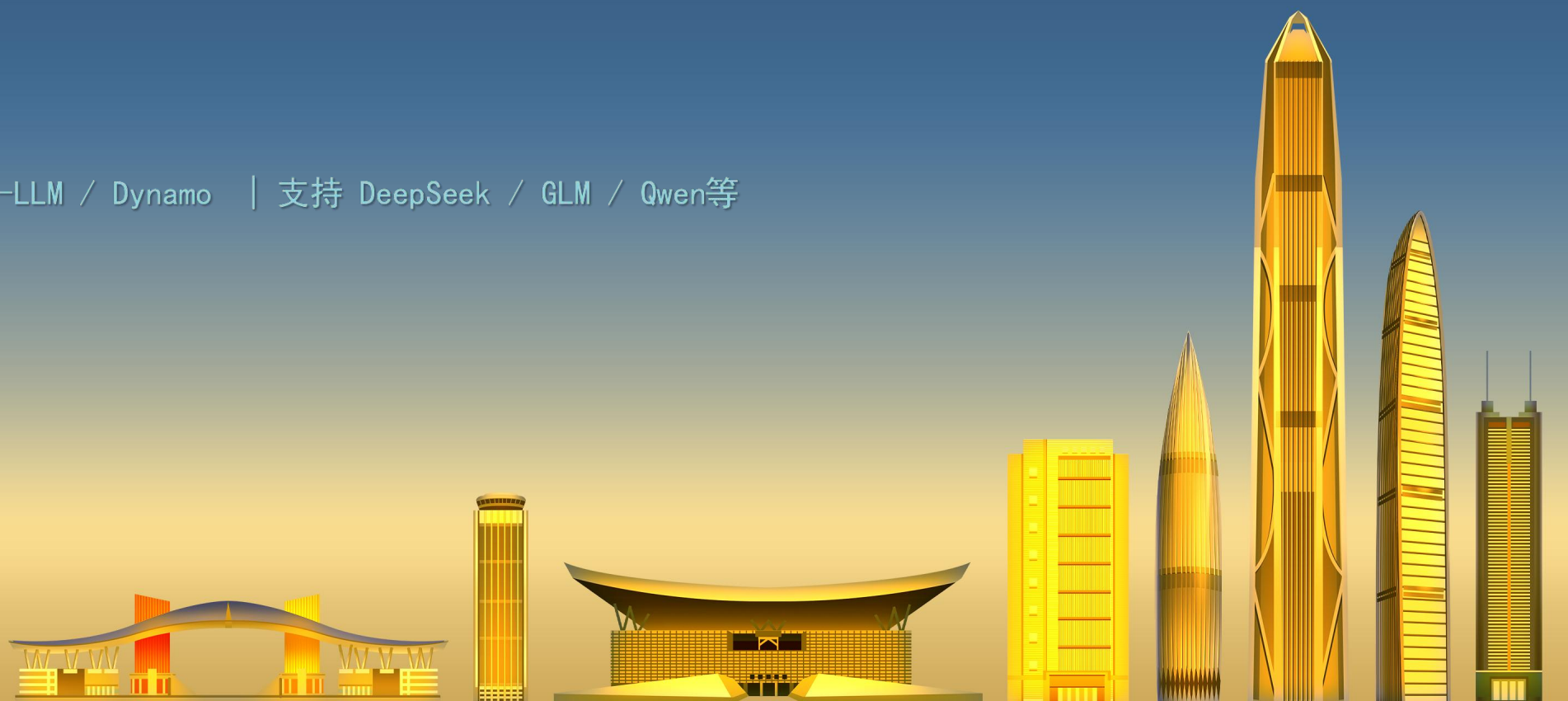
cache_salt 按会话划分缓存
同模板不同会话，命中行为精确可控

◆ 支持模型 / LoRA 版本切换与秒级回滚：新版本独立 namespace 上线，异常即切回

03

生态与集成

已合入 vLLM / SGLang / TRT-LLM / Dynamo | 支持 DeepSeek / GLM / Qwen等



开源与社区集成：一次接入，多框架复用

vLLM

- **v0.17.2** 起内置 FlexKVConnectorV1
- **无需手动打 patch**，原生启用
- 支持 dp>1、TP16、**NVFP4 无损卸载**

SGLang

- **v0.5.16** 起原生内置 backend
- **--enable-flexkv** 一键开启 CPU / SSD 卸载
- **DeepSeek-V4** 全特性适配

TensorRT-LLM

- **TP16** 大规模并行支持
- **Namespace 隔离**能力同步支持

NVIDIA Dynamo

- 原生 **KV Cache Offloading** 方案
- **KV 感知路由** + 多级缓存联合部署
- 多实例共享缓存**开箱即用**

◆ **全部合入上游主干：无 fork、无 patch 维护负担，版本升级即享新能力**

主流模型支持：从通用架构到最复杂的 DeepSeek-V4

DeepSeek 全系列（含 V4 最新特性）

- 异构 **C4 / C128 / indexer** KV group 分组管理
- **FullKV + SWA 双缓存**；attention / indexer compress-state sidecar
- **逐层恢复**（layerwise restore）+ SWA / state H2D 融合流水线

GLM / Qwen 等主流模型

- **Dense / MoE** 主流架构全覆盖
- 新模型适配**只需声明 KV 布局**

三种 KV 布局原生兼容

- vLLM / SGLang / TRT-LLM 布局**自动适配**
- **BLOCKFIRST / LAYERFIRST** 可选并自动选路

MLA 专项优化

- D2H 分片写入：**sharded 等 5 种模式**
- **rank 轮换**写入避免单卡热节流

一套完整的 KVCache 管理解决方案

为什么	Agent 场景下 KVCache 爆发增长，HBM 装不下；“分级存储 + 池化共享”是必然方向
怎么做	五大核心能力：多级存储管理 · KV 压缩 · 分布式池化 · 高效传输 · Namespace 隔离
用起来	已合入 vLLM / SGLang / TRT-LLM / Dynamo ，库函数形态低开销接入，支持 DeepSeek / GLM / Qwen等主流模型

TTFT：60% ↓ TPOT：13% ↓ QPM：16% ↑

GOSIM

GOSIM Shenzhen 2026

Thanks!

FlexKV：代码已开源，欢迎您的参与

GitHub: github.com/taco-project/FlexKV | Apache-2.0 开源

